شرح معمق للمكتبات المستخدمة في مشروع مساعد Gemini AI

يهدف هذا التقرير إلى تقديم شرح موسوعي ومفصل لكل مكتبة برمجية تم استخدامها في كود مساعد Gemini AI، مع التركيز على وظائفها الأساسية، آليات عملها الداخلية، ودورها الحيوي في بناء هذا التطبيق. سنغوص في التفاصيل التقنية لكل مكتبة لنفهم القوة الكامنة وراء كل سطر برمجي.

1. مكتبة `enum`

على الرغم من أن مكتبة `enum` تم استيرادها في الكود المرفق (`import enum`)، إلا أنها لم تُستخدم بشكل مباشر في الأجزاء المقدمة. ومع ذلك، فإن فهمها ضروري لأي مطور بايثون.

1.1. ما هي `enum` ؟

التعداد هو مجموعة من **تعدادات (enumerations)**. `enum` هي وحدة قياسية في بايثون توفر دعمًا لإنشاء الأسماء الرمزية (أعضاء) المرتبطة بقيم فريدة وثابتة. تُستخدم التعدادات لجعل الكود أكثر قابلية للقراءة والصيانة عن طريق استبدال القيم السحرية (magic numbers/strings) بأسماء ذات معنى.

1.2. لماذا نستخدم التعدادات؟

- **قابلية القراءة:** بدلاً من استخدام `if status == 0:` ، يمكن استخدام `if status == StatusCode.SUCCESS:` . هذا يجعل الكود أسهل في الفهم.
- **تجنب الأخطاء:** تمنع الأخطاء الإملائية أو استخدام قيم غير صالحة، حيث أن أعضاء التعداد معرفون مسبقًا.
- **الصيانة:** إذا تغيرت القيمة الأساسية لعضو التعداد، يكفي تغييرها في مكان واحد فقط.

1.3. مثال على الاستخدام (غير موجود في الكود المرفق)

Python

```python
from enum import Enum

class TrafficLight(Enum):
    RED = 1
    YELLOW = 2
    GREEN = 3

current_light = TrafficLight.RED
if current_light == TrafficLight.RED:
    print("Stop!")
```

2. مكتبة (Natural Language Toolkit) nltk

توفر أدوات قوية للعمل مع البيانات النصية، من التحليل الأساسي إلى بناء نماذج معقدة. في هذا المشروع، تُستخدم NLTK في مراحل ما قبل المعالجة (preprocessing) للنصوص.

2.1. تحميل بيانات (nltk.download , nltk.data.find) NLTK

- **الآلية:** قبل استخدام بعض وظائف NLTK ، يجب تنزيل مجموعات بيانات أو نماذج معينة. يقوم الكود بالتحقق مما إذا كانت بيانات punkt و punkt_tab موجودة باستخدام nltk.data.find() . إذا لم يتم العثور عليها، يتم تنزيلها تلقائيًا باستخدام nltk.download() .
- **الأهمية:** punkt هو مُقسم جمل (sentence tokenizer) مدرب يستخدم خوارزمية غير خاضعة للإشراف لتقسيم النص إلى قائمة من الجمل. هذا يضمن أن أدوات NLTK الأخرى (مثل word_tokenize) تعمل بشكل صحيح عن طريق توفير البيانات اللازمة.

2.2. تقسيم الكلمات (nltk.word_tokenize)

- **الآلية:** تقوم الدالة nltk.word_tokenize() بتقسيم سلسلة نصية (جملة) إلى قائمة من الكلمات أو الرموز (tokens). تستخدم هذه الدالة نموذج punkt المدرب لتحديد حدود الكلمات وعلامات الترقيم.
- **كيف تعمل داخليًا:** تعتمد word_tokenize على خوارزمية Punkt لتقسيم الجمل أولاً، ثم تقسيم كل جملة إلى كلمات. تأخذ في الاعتبار قواعد اللغة لتحديد ما إذا كانت النقطة نهاية جملة أم جزءًا من اختصار (مثل Dr.).
- **الدور في المشروع:** هي الخطوة الأولى لتحويل نص المستخدم الخام إلى تنسيق يمكن للنموذج فهمه ومعالجته. بدون تقسيم الكلمات، سيكون من المستحيل تحليل الجملة بشكل فعال.

2.3. تصغير الكلمات (LancasterStemmer)

- **الآلية:** LancasterStemmer هي خوارزمية لتصغير الكلمات (stemming) تقوم بتقليل الكلمات إلى جذرها اللغوي أو شكلها الأساسي عن طريق إزالة اللواحق. على سبيل المثال، تحول "connection", "connecting", "connected" إلى "connect".
- **كيف تعمل داخليًا:** تعتمد LancasterStemmer على مجموعة من القواعد (rules) المحددة مسبقًا لإزالة اللواحق من الكلمات. هذه القواعد يتم تطبيقها بشكل متكرر حتى لا يمكن تطبيق المزيد من القواعد أو حتى يتم الوصول إلى جذر الكلمة. إنها خوارزمية قوية ولكنها قد تكون عدوانية بعض الشيء، مما يعني أنها قد تقطع الكلمات أكثر من اللازم في بعض الأحيان.
- **الدور في المشروع:**
- **توحيد الكلمات:** تضمن أن الأشكال المختلفة لنفس الكلمة (مثل "run", "running", "runs") يتم التعامل معها ككيان واحد، مما يقلل من حجم المفردات ويحسن قدرة النموذج على التعميم.
- **تقليل الأبعاد:** في تمثيل "كيس الكلمات" (Bag of Words)، يقلل التصغير من عدد الميزات الفريدة، مما يجعل النموذج أبسط وأسرع في التدريب.

3. مكتبة `numpy`

هي مكتبة أساسية للحوسبة العلمية في بايثون، وتوفر كائن مصفوفة قويًا متعدد الأبعاد `ndarray`، بالإضافة إلى أدوات للعمل مع هذه المصفوفات. إنها حجر الزاوية للعديد من مكتبات التعلم الآلي والبيانات.

3.1. ما هي `numpy` ؟

- **المصفوفات (Arrays):** الكائن الأساسي في `NumPy` هو `ndarray`، وهو مصفوفة متجانسة من العناصر (جميعها من نفس النوع) يمكن أن تكون بأي عدد من الأبعاد. هذه المصفوفات أكثر كفاءة في التخزين والمعالجة من قوائم بايثون العادية، خاصة للعمليات العددية الكبيرة.
- **العمليات المتجهة (Vectorized Operations):** تتيح `NumPy` إجراء عمليات رياضية على المصفوفات بأكملها دفعة واحدة (vectorization) بدلاً من التكرار على العناصر الفردية. هذا يؤدي إلى كود أكثر إيجازًا وأسرع بكثير.

3.2. الدور في المشروع (`import numpy as np`)

- **تحويل البيانات:** تُستخدم `NumPy` لتحويل قوائم بايثون العادية (مثل `training` و `output` بعد معالجة البيانات) إلى مصفوفات (`np.array()`). هذا ضروري لأن مكتبة `TensorFlow` تتوقع مدخلات في شكل مصفوفات `NumPy` أو `Tensor`.
- **العمليات العددية:** تُستخدم `NumPy` أيضًا في دالة `bag_of_words` لإنشاء متجه "كيس الكلمات"، وفي معالجة مخرجات النموذج (مثل `np.argmax()`) للعثور على الفهرس ذي القيمة القصوى).

4. مكتبة `tensorflow` (مع Keras)

هي مكتبة مفتوحة المصدر رائدة لتطوير ونشر نماذج التعلم الآلي، وخاصة التعلم العميق. `TensorFlow` 2 البرمجية عالية المستوى، والتي تبسط عملية بناء `Keras` في هذا الكود، يتم استخدامها من خلال واجهة وتدريب الشبكات العصبية.

4.1. ما هو `TensorFlow` و `Keras` ؟

- **TensorFlow:** إطار عمل شامل للتعلم الآلي يوفر أدوات لبناء الرسوم البيانية الحسابية، وإجراء العمليات على الـ Tensors (كائنات بيانات متعددة الأبعاد)، وتدريب النماذج على وحدات المعالجة (CPUs) المركزية ووحدات معالجة الرسوميات (GPUs).
- **Keras:** واجهة برمجة تطبيقات (API) عالية المستوى تعمل فوق `TensorFlow` (أو أطر عمل أخرى). `Keras` تجعل بناء وتدريب نماذج التعلم العميق أسهل وأسرع بكثير من استخدام `TensorFlow` الخام.

4.2. بناء النموذج (`tf.keras.Sequential`)

- **`tf.keras.Sequential` :** يسمح بإنشاء نموذج عن طريق تكديس `Keras`. هو أبسط أنواع النماذج في الطبقات بشكل خطي، طبقة تلو الأخرى. هذا مناسب للشبكات العصبية ذات التغذية الأمامية (feed-forward neural networks) مثل تلك المستخدمة هنا.

4.3. الطبقات (tf.keras.layers.Dense)

- **tf.keras.layers.Dense :** تمثل طبقة متصلة بالكامل (fully connected layer) كل طبقة كثيفة. أو وحدة عصبية في هذه الطبقة متصلة بكل وحدة عصبية في الطبقة السابقة.
  - **units (عدد الوحدات العصبية):** يحدد عدد الخلايا العصبية في الطبقة. في هذا الكود، الطبقات المخفية تحتوي على 8 وحدات عصبية، وطبقة الإخراج تحتوي على عدد وحدات عصبية يساوي عدد النوايا (التصنيفات).
  - **input_shape :** يُستخدم في الطبقة الأولى لتحديد شكل المدخلات المتوقعة للنموذج. في هذا المشروع، هو طول متجه "كيس الكلمات".
  - **activation (دالة التنشيط):** تحدد دالة غير خطية يتم تطبيقها على مخرجات كل وحدة عصبية. تضيف اللاخطية إلى النموذج، مما يسمح له بتعلم علاقات معقدة.
    - **relu (Rectified Linear Unit):** دالة تنشيط شائعة للطبقات المخفية. تُرجع المدخلات إذا كانت موجبة، وصفرًا إذا كانت سالبة. بسيطة وفعالة وتساعد في حل مشكلة التدرج المتلاشي [2].
    - **softmax :** دالة تنشيط تُستخدم عادة في طبقة الإخراج لمهام التصنيف متعدد الفئات. تحول مخرجات الطبقة إلى توزيع احتمالي، حيث يكون مجموع الاحتمالات لجميع الفئات 1. هذا يسمح بتفسير المخرجات كاحتمالية انتماء الإدخال لكل فئة [2].

4.4. تجميع النموذج (model.compile)

- **optimizer (المحسن):** adam [2]. هو خوارزمية تحسين شائعة وفعالة تعتمد على التدرج التنازلي التكيفي. يقوم بضبط معدل التعلم لكل معلمة في النموذج بشكل مستقل، مما يؤدي إلى تقارب أسرع وأكثر استقرارًا أثناء التدريب.
- **loss (دالة الخسارة):** categorical_crossentropy [2]. تُستخدم هذه الدالة لقياس مدى جودة أداء النموذج في مهمة التصنيف متعدد الفئات عندما تكون التسميات في شكل one-hot encoded. الهدف هو تقليل قيمة دالة الخسارة أثناء التدريب.
- **metrics (المقاييس):** accuracy [2]. هو مقياس لتقييم أداء النموذج، حيث يقيس نسبة التنبؤات الصحيحة. يُستخدم لمراقبة تقدم التدريب ولكن لا يؤثر بشكل مباشر على عملية التحسين.

4.5. تدريب النموذج (model.fit)

- **output و training :** هما بيانات الإدخال والمخرجات المستهدفة التي يتعلم منها النموذج.
- **epochs :** عدد المرات التي يمر فيها النموذج عبر مجموعة بيانات التدريب بأكملها. كلما زاد عدد الـ epochs، زادت فرصة النموذج للتعلم، ولكن قد يؤدي ذلك إلى الإفراط في التخصيص (overfitting).
- **batch_size :** عدد العينات التي يتم معالجتها قبل تحديث أوزان النموذج. حجم الدفعة الصغير (8 في هذا الكود) يعني تحديثات متكررة للأوزان.

4.6. حفظ وتحميل النموذج (tf.keras.models.load_model , model.save)

- **`model.save("chatbot_model.h5")` :** يقوم بحفظ النموذج المدرب بالكامل (بما في ذلك بنيته وأوزانه وحالة المحسن) في ملف بصيغة HDF5. هذا يسمح بإعادة استخدام النموذج لاحقًا دون الحاجة إلى إعادة تدريبه.
- **`tf.keras.models.load_model("chatbot_model.h5")` :** يقوم بتحميل نموذج مدرب مسبقًا من ملف HDF5. هذا يوفر وقتًا كبيرًا في التطبيقات التي تحتاج إلى استخدام النموذج بشكل متكرر.

5. مكتبة `random`

`random` هي وحدة قياسية في بايثون توفر دوال لتوليد أرقام عشوائية واختيار عناصر عشوائية من القوائم.

5.1. الدور في المشروع (`random.choice`)

- **`random.choice(sequence)` :** تُستخدم هذه الدالة لاختيار عنصر عشوائي واحد من قائمة (sequence) معينة. في هذا المشروع، بعد أن يحدد النموذج نية المستخدم، يتم استخدام `random.choice()` لاختيار رد عشوائي من قائمة الردود المتاحة لتلك النية. هذا يضيف تنوعًا إلى استجابات الروبوت ويجعله يبدو أكثر طبيعية وأقل تكرارًا.

6. مكتبة `json`

`json` هي وحدة قياسية في بايثون توفر أدوات للتعامل مع بيانات JSON (JavaScript Object Notation)، وهو تنسيق شائع لتبادل البيانات.

6.1. الدور في المشروع (`json.load`)

- **`json.load(file_object)` :** تُستخدم هذه الدالة لقراءة بيانات JSON من كائن ملف (file object) وتحويلها إلى كائن بايثون (عادةً قاموس أو قائمة). في هذا المشروع، يتم استخدامها لتحميل ملف `intent.json`، الذي يحتوي على تعريفات النوايا، الأنماط، والردود الخاصة بالروبوت.
  - **الأهمية:** يتيح `json` تخزين بيانات التكوين (مثل النوايا والردود) في ملف خارجي منظم وسهل القراءة، مما يفصل البيانات عن الكود ويجعل من السهل تحديثها أو تعديلها دون الحاجة إلى تغيير الكود البرمجي الأساسي.

7. مكتبة `pickle`

`pickle` هي وحدة قياسية في بايثون لتسلسل (serialize) وإلغاء تسلسل (deserialize) كائنات بايثون. التسلسل هو عملية تحويل كائن بايثون إلى تدفق بايت (byte stream) يمكن تخزينه في ملف أو نقله عبر الشبكة. إلغاء التسلسل هو العملية العكسية.

7.1. الدور في المشروع (`pickle.load` , `pickle.dump`)

- **`pickle.dump(obj, file_object)` :** تُستخدم لحفظ كائن بايثون (`obj`) في ملف (`file_object`) بتنسيق ثنائي. في هذا المشروع، يتم استخدامها لحفظ البيانات المعالجة مسبقًا (`words` , `labels` , `training` , `output`) في ملف `data.pickle` .

- **`pickle.load(file_object)` :** تُستخدم لتحميل كائن بايثون من ملف ثنائي. في هذا المشروع، يتم استخدامها لتحميل البيانات المعالجة من `data.pickle`.
- **الأهمية:** تتيح `pickle` استمرارية البيانات. بدلاً من إعادة معالجة النصوص من ملف `intent.json` وتوليد مصفوفات التدريب في كل مرة يتم فيها تشغيل التطبيق (وهي عملية قد تستغرق وقتًا)، يمكن حفظ هذه البيانات المعالجة وتحميلها بسرعة، مما يوفر وقتًا كبيرًا ويحسن كفاءة بدء تشغيل التطبيق.

8. وحدة `time` (الدالة `sleep`)

هي الأكثر صلة بهذا المشروع `sleep` هي وحدة قياسية في بايثون توفر دوال للتعامل مع الوقت. الدالة `time`.

8.1. الدور في المشروع (`time.sleep`)

- **`time.sleep(seconds)` :** توقف تنفيذ البرنامج مؤقتًا لعدد محدد من الثواني. في هذا المشروع، تُستخدم `sleep(0.4)` لإضافة تأخير بسيط (0.4 ثانية) قبل عرض رد المساعد الذكي.
- **الأهمية:** هذا التأخير الصغير يحاكي تجربة محادثة أكثر طبيعية، حيث لا تستجيب الروبوتات عادة بشكل فوري تمامًا. إنه تفصيل صغير ولكنه يساهم في تحسين تجربة المستخدم من خلال جعل التفاعل يبدو أقل "آليًا" وأكثر "إنسانية".

9. مكتبة `streamlit`

هي مكتبة مفتوحة المصدر لإنشاء تطبيقات ويب تفاعلية مخصصة لعلوم البيانات والتعلم `Streamlit` `4` الآلي، باستخدام بايثون فقط.

9.1. فلسفة `Streamlit`

- **بايثون فقط:** تتيح للمطورين بناء واجهات مستخدم معقدة دون الحاجة إلى كتابة HTML، CSS، أو JavaScript. يتم تحويل كود بايثون مباشرة إلى عناصر واجهة مستخدم تفاعلية.
- **التطوير السريع:** مصممة لتسريع عملية تحويل النصوص البرمجية (scripts) إلى تطبيقات ويب قابلة للمشاركة.
- **إعادة التشغيل التلقائي:** عندما يتم تعديل الكود، يقوم `Streamlit` تلقائيًا بإعادة تشغيل التطبيق وتحديث الواجهة.

9.2. المكونات الرئيسية في المشروع

- **`st.set_page_config()` :** تُستخدم لتكوين إعدادات صفحة الويب، مثل العنوان (`page_title`)، الأيقونة (`page_icon`)، وتخطيط الصفحة (`layout`).
- **`st.sidebar` :** يسمح بإنشاء شريط جانبي في التطبيق، وهو مكان مثالي لوضع عناصر التحكم أو المعلومات الثانوية (مثل اختيار الوضع الداكن/الفاتح في هذا الكود).
- **`st.radio()` :** ينشئ مجموعة من أزرار الاختيار (radio buttons) التي تسمح للمستخدم باختيار خيار واحد من بين عدة خيارات. تُستخدم هنا لاختيار الوضع الداكن أو الفاتح.

- **st.markdown() :** تُستخدم لعرض نص بتنسيق Markdown. يمكن أيضًا استخدامها لحقن كود HTML/CSS خام في التطبيق (مع `unsafe_allow_html=True`)، وهو ما تم استخدامه لتخصيص مظهر التطبيق بشكل كبير (مثل ألوان الخلفية، الخطوط، وتأثيرات العنوان).
- **st.write() :** دالة متعددة الاستخدامات لعرض النصوص، البيانات، الرسوم البيانية، وغيرها من العناصر في التطبيق.
- **st.session_state :** ميزة حيوية في `Streamlit` تسمح بتخزين البيانات عبر عمليات إعادة تشغيل التطبيق وتفاعلات المستخدم. في هذا المشروع، تُستخدم لتخزين سجل المحادثة (`st.session_state.message`)، مما يسمح للروبوت بتذكر الرسائل السابقة والحفاظ على سياق المحادثة.
- **st.chat_input() :** ينشئ حقل إدخال مخصص للدردشة، وهو مثالي لتطبيقات الروبوتات المحادثة. يلتقط إدخال المستخدم ويعيد تشغيل جزء من الكود لمعالجته.
- **st.chat_message() :** تُستخدم لعرض رسائل الدردشة في واجهة مستخدم تشبه المحادثة، مع دعم للأفاتار (avatars) وأدوار المستخدم/المساعد.
- **st.spinner() :** تعرض مؤشر تحميل مرئي للمستخدم أثناء تنفيذ عملية تستغرق وقتًا، مما يحسن تجربة المستخدم عن طريق إظهار أن التطبيق لا يزال يعمل.

9.3. تخصيص المظهر (CSS Injection)

- يُظهر الكود استخدامًا متقدمًا لـ `st.markdown()` لحقن كود CSS مخصص. هذا يسمح بتغيير الألوان، الخطوط، الرسوم المتحركة، وتصميم عناصر واجهة المستخدم لتتناسب مع "Gemini Style"، مما يمنح التطبيق مظهرًا فريدًا وجذابًا. هذه المرونة تجعل `Streamlit` خيارًا قويًا لإنشاء تطبيقات ذات علامة تجارية خاصة.

خاتمة

من خلال هذا الشرح المعمق، نأمل أن تكون قد اكتسبت فهمًا شاملاً لكل مكتبة مستخدمة في مشروع مساعد Gemini AI. كل مكتبة تلعب دورًا محددًا وحاسمًا، وتكاملها معًا هو ما يمنح التطبيق قدرته على معالجة اللغة، التعلم، والتفاعل مع المستخدمين عبر واجهة ويب جذابة. هذا النهج يمثل نموذجًا قويًا لبناء تطبيقات الذكاء الاصطناعي التفاعلية بكفاءة وفعالية.

المراجع

[1] NLTK (Natural Language Toolkit). (n.d.). Retrieved from

[2] TensorFlow. (n.d.). Retrieved from

[3] NumPy. (n.d.). Retrieved from

[4] Streamlit. (n.d.). Retrieved from